

从模型诞生到上线： Ray 在小红书 AI 数据生 产中的算力调度实践

演讲人：陈 宇

小红书 / 数据引擎开发工程师

AICon
全球人工智能开发与应用大会



Speaker

陈宇

小红书·数据引擎团队
数据开发工程师

小红书数据引擎开发工程师, Ray/ Flink/Daft Contributor

专注于以 Ray 为统一底座, 为 AI 数据生产构建海量、好用、易用的算力供给体系 —— 覆盖训练数据处理、模型后训练、离线批量推理到近线实时推理的全链路。



目录

Agenda

01

开篇：为什么一套底座要覆盖全生命周期

02

Ray 底座能力：统一调度 + 丰富生态

03

训练数据准备：多引擎协同

04

离线刷库：弹性算力的多代演进

05

近线增量入库：自研流批引擎 Ray Klein

06

总结：算力供给模式必须匹配业务阶段

一条主线

数据生产贯穿 模型全生命周期

同一套 Ray 底座，
按阶段差异化供给。

01

开篇

AI数据生产伴随模型的整个生命周期



长在多云环境下的小红书

海量内容数据 · 多云多地域 · 分散资源

Billion

图文笔记
社区内容理解

10 Billion

视频抽帧
多模态处理

30 Billion

物品框
电商商品理解

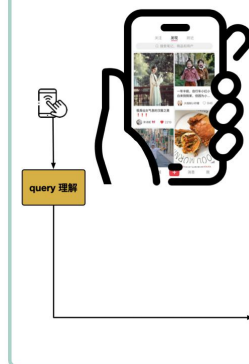
多云环境

- 业务长在多云 / 多地域之上
- 云原生部署，资源天然分散
- 10w+ node 碎片“沙里淘金”

业务场景

- 社区搜索 / 商品搜索 / 内容理解
- 笔记 Embedding / 标签 / 特征

简化后搜索过程



内容理解的产出 广泛应用于在线搜索：

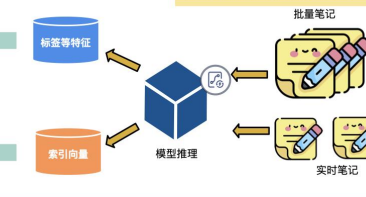
- Embedding 向量用于召回
- 产出标签等特征用于排序

两类计算：

计算量大：几十亿笔记

- 离线批量推理 -> 老笔记结果更新
- 近线实时推理 -> 新笔记结果生成

实时性高：秒级别延迟



搜索 / 内容理解：query → 召回 → 排序 → 结果展示

AI 数据生产，贯穿模型研发的完整生命周期

四个阶段，四种截然不同的算力诉求

模型生命周期



核心判断：没有一种供给模式能一劳永逸 —— 算力供给必须匹配业务阶段的特征。

02

Ray 底座能力

统一调度 + 丰富生态，一套底座的两块基石



Ray Core: 把一个集群变成一台“超级计算机”



全局资源视图，让调度与通信不再是业务的负担

全局资源视图

调度器统一掌握集群 CPU/GPU/内存，
按需把任务放到最合适的节点

任意节点发起

任一节点都能发起分布式任务，
无中心瓶颈，开发即本地体验

高性能通信

小数据直传、大数据共享内存零拷贝，
跨节点搬运开销大幅降低

AutoScaler

按真实负载自动扩缩容，
无需提前规划资源水位



丰富生态：让“合适的引擎”做“合适的事”

底座之上，按场景自由组合上层框架



Ray Data

通用数据并行处理



Ray Serve

在线 / 离线模型服务



Ray Klein

自研流批统一引擎



Daft

多模态高效格式解析



RayDP

Spark on Ray · 大规模 Shuffle



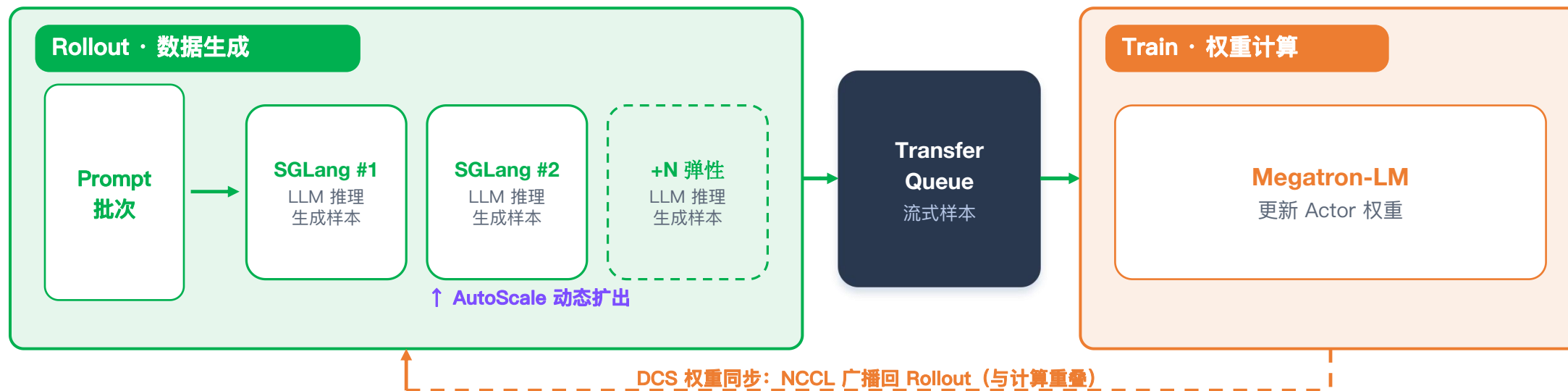
DataJuicer

数据质量清洗与过滤

关键：业务只写一个脚本，引擎切换由底座统一承接 —— 框架是手段，统一调度才是底座的价值。

延伸：RL 后训练 —— Ray 调度 Actor + 弹性 Rollout

Rollout 占 50—60% 时间成瓶颈，Relax 基于 Ray 动态扩容



Ray 底座 · Actor 进程调度 + AutoScale 按需扩缩 Rollout 节点

VeRL / Slime / Relax (小红书自研) 等 RL 引擎均 on Ray 调度

模型越大 Rollout 越是瓶颈 —— 用 Ray AutoScale 动态加推理副本，把这一步的墙钟时间压下来。

03

训练数据准备

多引擎协同：稳定常驻集群 + 异构处理链路



数据画像 · 训练数据准备

先看清这批数据是什么、从哪来、要扛多大计算压力

用途

预训练语料
SFT 精调数据

模型越大，对语料规模与
算力的需求越大

来源

自采集 + 采买
多模态原始数据

数据源复杂、格式多样，
加工链路长且定制化高

计算压力

海量规模 · 多引擎
资料 ⇒ Token

去重 / 打分 / 解析异构，
管线长、需分阶段落盘

核心挑战：数据管线长且天然异构，没有万能引擎，引擎间协同的成本极高。

挑战：预训练 / SFT 数据链路天然异构

没有万能引擎，而引擎间协同的成本极高

大规模去重

依赖海量 Shuffle
→ 需要 Spark 式算子

适配：RayDP

AI 质量打分

依赖 GPU 批量推理
→ 需要高吞吐推理

适配：Ray Data

多模态解析

图文/视频格式复杂
→ 需要高效格式读取

适配：Daft

一条数据处理链路，往往同时需要上面三类能力。

若每种引擎各起一个集群，数据要反复落盘搬运、资源彼此割裂 —— 协同成本吞掉了引擎本身的收益。

方案：一份资源驱动多引擎，业务自由互转

同一份数据在引擎间零拷贝流转，屏蔽底层资源



一个 Python 脚本，引擎间零拷贝自由互转

`df = spark.createDataFrame(...) → ds = ray.data.from_spark(df) → daft.from_ray_dataset(ds)`



RayDP

去重 Shuffle



Ray Data

GPU 批量打分



Daft

多模态解析



DataJuicer

质量清洗

RuntimeEnv 沙箱隔离 · 每个算子独立依赖，互不冲突



Ray 统一底座 · 全局调度 · 共享内存零拷贝



Kubernetes 异构资源 · 万核百卡 CPU / GPU

为 Agent 屏蔽资源：只需学会“提交 RayJob”

Agent 编排复杂数据管线，底层资源只看见一个 Ray

若让 Agent 直接面对多引擎多平台

- 学 Spark 怎么在 A 平台提交
- 学 Ray Data 怎么在 B 平台提交
- 学 Daft / Flink 各自的提交方式
- 还要各自处理资源、配额、调度

→ 学习成本高、易出错



只需学会一件事

把 RayJob 提交到 RayCluster

ray job submit ...

- 引擎组合在脚本里，Agent 只管编排逻辑
- 资源 / 多云 / 调度全部由 Ray 屏蔽
- 大幅降低 Agent 学习与试错成本

随 Agent coding 能力增强，用 Ray 开发复杂数据管线、同时为 Agent 屏蔽资源 —— 它只看见一个 Ray。



工程实践：从“跑得起来”到“稳得住”

万核百卡集群暴露的真问题，比 Demo 难得多

本地直连集群开发

效率

开发态直接连大集群调试，
所见即所得，迭代效率大幅提升

RuntimeEnv 依赖隔离

隔离

多引擎依赖版本冲突，
用沙箱把环境问题挡在算子边界内

GCS 稳定性

稳定

全局控制面在大规模下成为瓶颈，
做高可用与压力治理

调度与日志治理

规模

海量 Task 的调度退化、日志风暴，
针对性优化可观测与吞吐

把“选引擎”的难题，变成 Ray 底座之上“组合引擎”的能力

1

资源策略

稳定常驻的异构集群，让多引擎长期共存

2

核心手段

RuntimeEnv 沙箱隔离，一个脚本无缝切换引擎

3

数据流转

在集群内存中流转、零拷贝，吸收协同成本

04

离线刷库

弹性算力：存量数据回扫的多代方案演进

数据画像 · 离线刷库

先看清这批数据是什么、从哪来、要扛多大计算压力

用途

**模型上线前
存量回扫入库**

新模型上线前，要用它把
历史存量数据全部重新打标 / 推理一遍

来源

**存量库表
历史笔记 / 商品**

已沉淀的全量业务数据，
一次性读取，不持续产生

计算压力

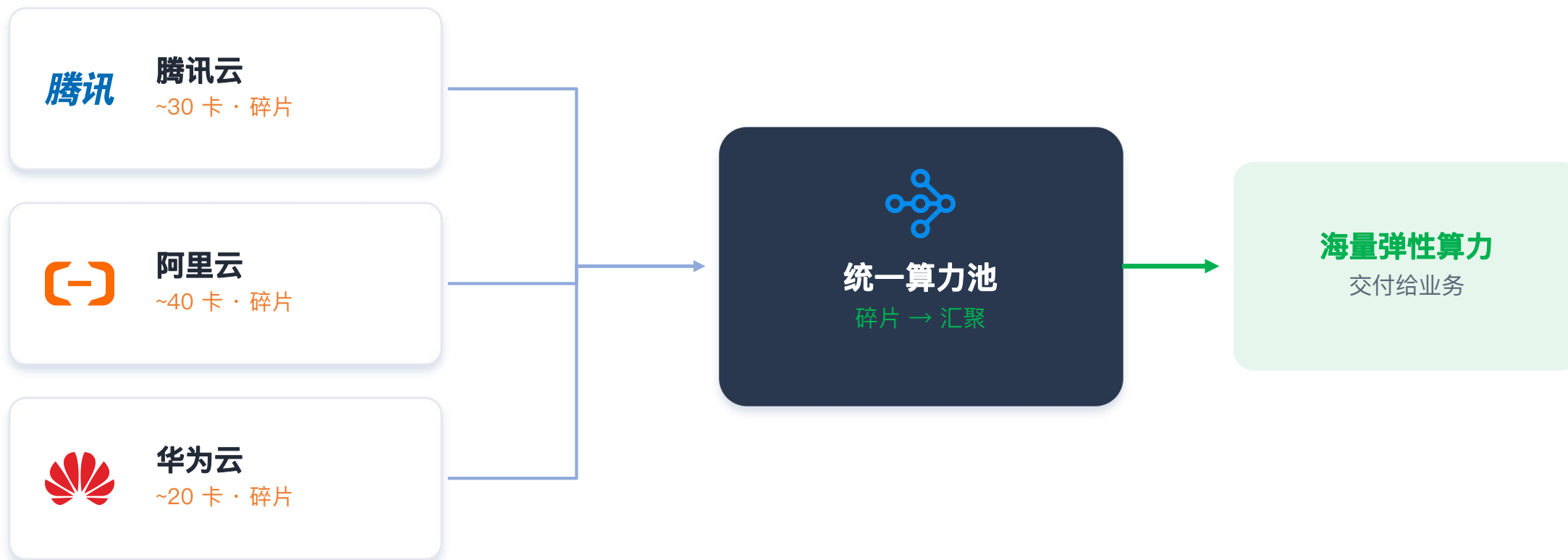
**数十亿行 · GPU 密集
来得急 · 一次性**

GPU 推理密集，资源离散在
多云多区，且随时被高优业务驱逐

由此带来的核心挑战：来得急、量级大、一次性，而 GPU 资源天然离散在多云多区。

GPU 困境：碎片资源散落多云，如何汇聚？

多云采买、每集群只有小几十卡、弹性资源高度分散



难题：账号 / 配额割裂、随时被高优业务驱逐 —— 如何把碎片汇聚成可用的海量算力？

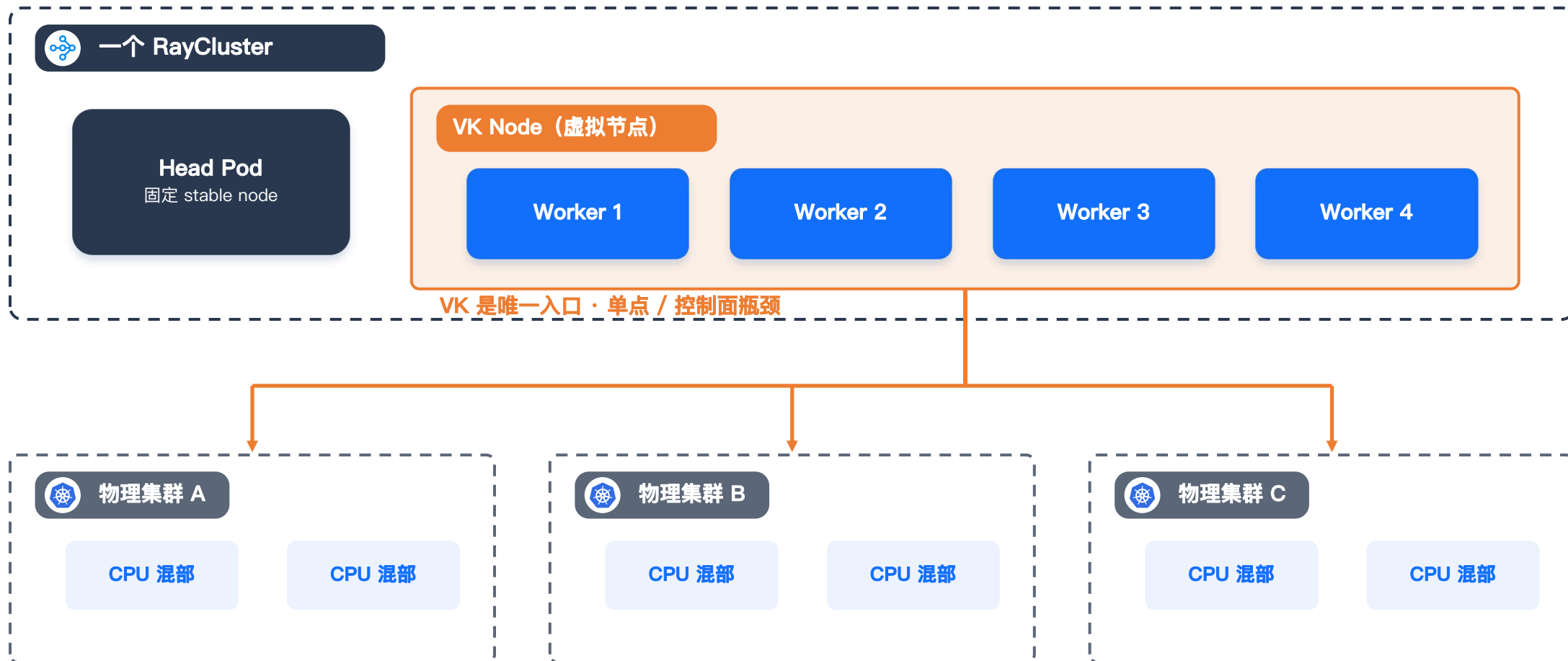
方案演进：从“聚合资源”到“分离架构”再到“联邦”

每一代都在解决上一代暴露的核心矛盾



第一代：Virtual Kubelet 聚合多集群

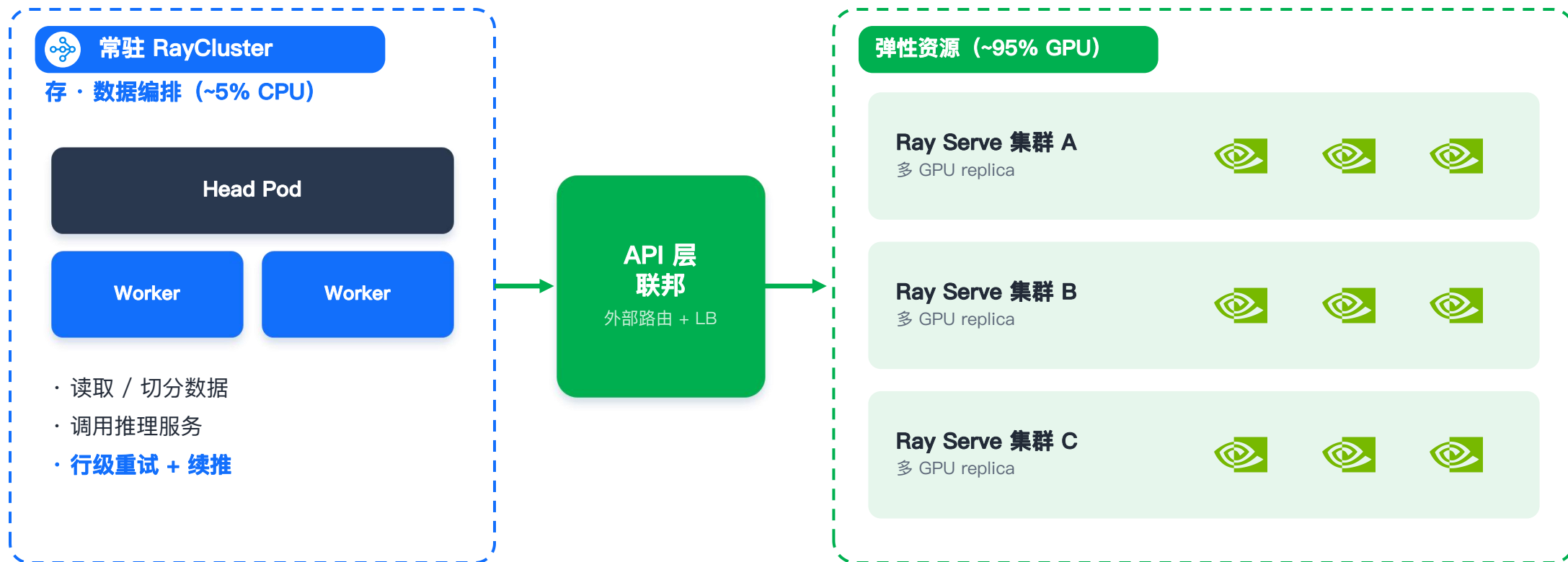
一个 RayCluster: head 固定, worker 经 VK 漂移到物理集群



✓ 跨云 CPU 混部资源首次聚合成统一入口 x VK 单点瓶颈 · 扩容/驱逐都要中转 · 10K→400K 核/时扛不住

第二代：存算分离 —— 用 5% 换 95%

常驻 RayCluster 做编排，API 层联邦驱动多个 Ray Serve



用 5% 常驻 CPU 换 95% 弹性 GPU —— 模型变成可独立伸缩的服务，GPU 被驱逐只需重试单行。

效果对比：长尾问题被大幅收敛

同一批存量回扫任务，两代方案的关键指标对比

维度	第一代 · Virtual Kubelet	第二代 · 存算分离
资源获取	整集群中转，扩容慢	客户端按需直连，秒级伸缩
驱逐影响	波及子任务，需大块重算	行级重试，影响收敛到单行
断点恢复	依赖任务级重跑	断点续推，已完成不重算
长尾表现	尾部子任务严重拖慢整体	长尾大幅收敛，完成更可预期

此外：我们正在向 KubeRay 开源社区提出 KubeRay Federation 方案（kuberay#4561）。



演进中：KubeRay 联邦 —— 调度下沉、分治弹性



Primary RayCluster 统一入口，各集群 RayCluster CR 自治 + AutoScale



业务视角：单一 RayCluster

Ray Core 无缝资源感知 · 无需单独部署 · Serverless scale 无感知

KubeRay 控制器 + Primary RayCluster (联邦入口)

多集群弹性资源统一调度



子集群 A

RayCluster CR · 自治

Worker

Worker

+N



GPU · AutoScale 动态扩缩



子集群 B

RayCluster CR · 自治

Worker

Worker

+N



GPU · AutoScale 动态扩缩



子集群 C

RayCluster CR · 自治

Worker

Worker

+N



GPU · AutoScale 动态扩缩

对比 VK：各子集群 RayCluster CR 自治、入口无单点压力、AutoScaler 按需扩缩 —— 够弹性 · #4561

选型：Ray Serve API 能取代联邦吗？

答案取决于负载——LLM 服务用 Serve，数据加工管道用联邦



Ray Serve API

适合：标准模型服务

✓ 典型 LLM 调用，简单且合适

- ✗ per-job 定制代码复杂、服务化成本高
- ✗ 调试要分别查客户端 / 服务端日志
- ✗ 依赖外部路由层 + LB 跨集群
- ✗ 存量代码改造成本、违背 Ray 简洁 API



KubeRay 联邦

适合：复杂数据加工管道

✓ 数据清洗 / 语料加工（资料 → Token）

- ✓ 异构计算分阶段、代码定制度高
- ✓ 单一 RayCluster 屏蔽 K8s / AZ 细节
- ✓ Serverless 弹性 scale 无感知
- ✓ 排查只需看 Ray 日志，无需双端

标准 LLM / 模型服务

→ 用 Ray Serve API

复杂 / 定制化数据加工管道

→ 用 KubeRay 联邦

弹性场景的胜负手，是快速汇集所有可用的资源

1

资源策略

海量弹性算力：把多云碎片快速汇聚成可用大池

2

方案演进

VK 聚合 → 存算分离驱动弹性 → KubeRay 联邦

3

关键收益

资源汇得越快越全，回扫的墙钟时间就越短

05

近线增量入库

流式算力：自研流批统一引擎 Ray Klein



数据画像 · 近线增量入库

先看清这批数据是什么、从哪来、要扛多大计算压力

用途

**新笔记 / 新商品
分钟级实时入库**

模型上线后，对持续产生的
新内容分钟级推理打标入库

来源

**Kafka 实时流
持续不断产生**

增量数据像水流一样持续到来，
而非一次性的存量库表

计算压力

**7× 24 · 流量波动
精确断点恢复**

需常驻在线、按流量弹性伸缩，
重启不丢不重保证一致

由此带来的核心挑战：分钟级时效、7× 24 在线、流量弹性，且重启要精确恢复不丢不重。

方案：自研流批统一引擎 Ray Klein

一套业务代码，在全量批推与近线增量之间无缝切换

一致性快照

基于 Chandy-Lamport 分布式快照
实现精确 Kafka offset 断点恢复

数据驱动 + 反压

按下游处理能力自动调速
动态平衡吞吐，避免堆积

全局重启 + 可观测

故障时全局一致重启
端到端指标可观测

流批一体

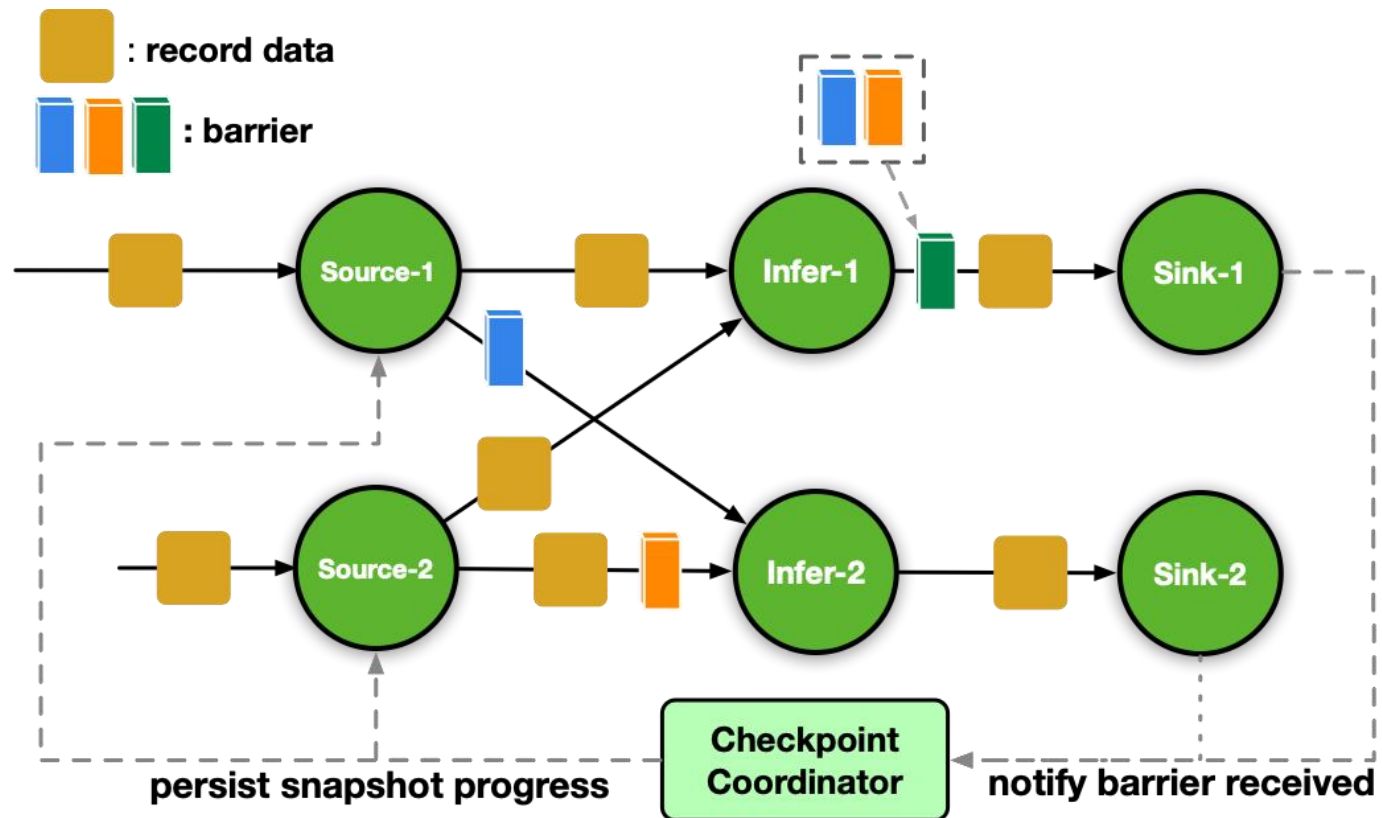
同一份算子代码
批模式回扫 / 流模式增量

需求：分钟级 · 7× 24 · 弹性 · 精确恢复 —— 离线“一次性”思路不再适用，需要长期在线的流式引擎。



架构：Ray Klein Streaming Checkpoint

借鉴 Chandy-Lamport: barrier 随数据流转, Coordinator 汇总快照



流批一体：一套代码，两种运行模式

Source 决定模式，同一份算子零改动



收益：开发一次、口径一致、维护减半 —— 近线与离线本质是同一份逻辑的两种数据来源。

让近线与离线收敛成同一件事 —— 一套代码，资源交给 Ray

1

资源策略

持续自适应的流式算力，按流量自动伸缩

2

核心手段

自研 Ray Klein + Chandy-Lamport
轻量 Checkpoint

3

关键收益

快速恢复，流批统一

06

总结

算力供给模式，必须匹配业务阶段



一张表看懂：三阶段的差异化供给

同一套 Ray 底座，三种资源策略与框架组合

阶段	算力诉求	资源策略	核心手段
训练数据准备	稳定常驻 + 多引擎	常驻异构集群	统一底座 + RuntimeEnv 协同
离线刷库	短期爆发 + 跨云	海量弹性 GPU	VK 聚合 → 存算分离 → Federation
近线增量入库	持续自适应 + 精确恢复	流式弹性算力	Ray Klein + 一致性快照

核心结论：算力供给模式必须匹配业务阶段的特征，而非用单一模式覆盖所有场景。

业务效果：以 Ray 为统一引擎底座

同一套底座，用不同框架组合与资源策略，驱动全场景

统一调度

全局资源视图
任意节点发起分布式任务

异构资源

CPU / GPU、多云多区
弹性与常驻统一纳管

生态丰富

上层框架开箱即用
Data / Serve / Klein / Daft ...

整体规模

上亿

核时 / 月
弹性 CPU

百万

卡时 / 月
GPU 算力

万核百卡

异构集群
稳定常驻

3

类场景
一套底座覆盖

后续展望

持续推进联邦、多模态上量，迈向 Ray Serverless

01

KubeRay 联邦

把跨集群统一调度做进框架层

Primary RayCluster + 各集群 CR 自治，回馈开源社区 (#4561)

02

Daft 多模态上量

把多模态格式解析推向更大规模

图文 / 视频 / 音频统一处理，支撑 Omni 模态数据加工

03

Ray Serverless 化

on Ray 算力按量计费、单一入口

彻底屏蔽多云 / 多 AZ / CPU·GPU，对客只有一个 Ray 集群

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

数新智能 / 创始人兼 CEO





TAKEAWAY

一套 Ray 底座， 让算力匹配数据生产的每个阶段。

供给模式匹配阶段特征，而非用单一模式削足适履 —— 这是我们做数据处理算力的核心判断。

THANKS

探索 AI 应用边界

Explore the limits of AI applications